

Advanced Frontend Course Architecture of Web Applications



Overview

Good architecture doesn't make your app faster for users. It makes development faster for team. It's about developer experience:

- How quickly can someone new contribute?
- How many files do you touch for a simple often do you ask "where does this go?"



Overview

Architecture is not about (not *only* about them). It's about:

- What parts your system consists of
- How these parts connect to each other
- What depends on what

*The relationships between parts dictate the folder structure
— not the other way around.*



Architecture is a process, not a one-time decision. Start simple. Evolve when you understand more.

The Process

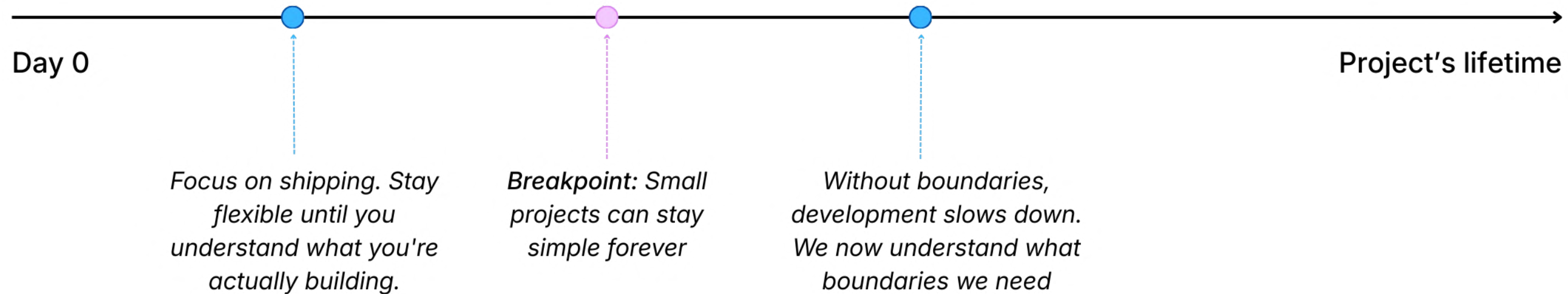
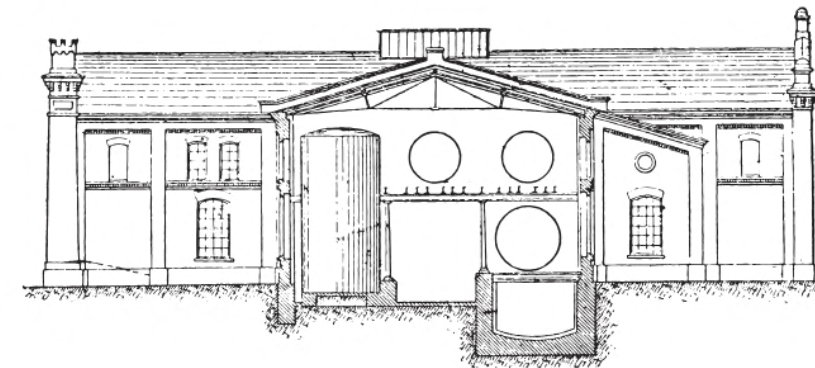
No structure



Minimal Architecture



Formalized Architecture





The Process

Build, Observe, Formalize

1. **Start Simple** - Minimal structure, just enough to start shipping
2. **Observe Patterns** - As you build, notice what code changes together, what's shared
3. **Identify Problems** - unclear boundaries, brittle code, difficult testing, or slow development.
4. **Formalize & Adapt** - Choose the architecture pattern that fits your problems, then adapt it to your needs—patterns are guidelines, *not rigid rules*.



How do you choose?

There's no universal answer. The right architecture depends on your context.

1. Project size
2. Project lifetime
3. Frontend domain complexity
4. New project vs. re-architecting
5. Tech stack

Let's look at each.



Project Size

Small project — simple structure is fine (follow LIFT principle).

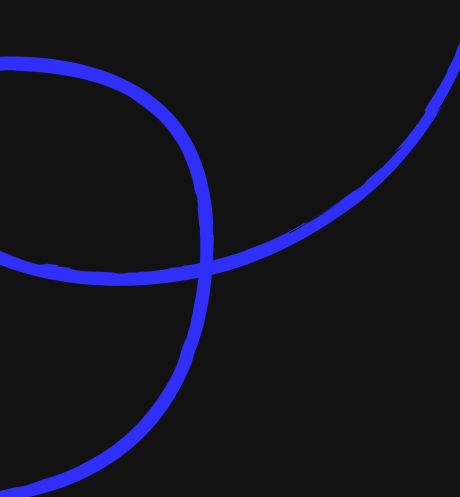
Patterns: Container/Presentational, Layered

Medium project — you need some organization.

Patterns: Layered, Vertical Slices, FSD

Large project — you need explicit structure. Feature boundaries, strict import rules, clear ownership.

Patterns: FSD, Clean Architecture, Vertical Slices



Locate code easily
Identify what it does at a glance
Flat structure as long as you can
Try to stay DRY



Project lifetime

MVP or prototype — prioritize speed and flexibility.

Minimal rules, easy to change, easy to throw away.

Patterns: Container/Presentational, Layered

Long-term product — invest in maintainability. Stricter rules, clear boundaries, good documentation.

Patterns: Layered, FSD, Clean Architecture, Vertical Slices



Frontend domain complexity

Thin frontend (logic lives on backend, or no heavy logic at all) — simple architecture.

Patterns: Container/Presentational, Layered, Vertical Slices

Complex frontend logic (calculations, validations, multi-step workflows in the browser) — isolate business logic from UI.

Patterns: Clean Architecture, FSD with isolated domain



New project vs. re-architecting

New project with unclear requirements — prefer flexibility. Simple structure that can evolve.

Patterns: Layered, Vertical Slices, Container/Presentational

New project with clear requirements — stricter structure pays off.

Patterns: depends on the project

Re-architecting — you can be strict. Pick architecture that solves your actual problems.

Patterns: Layered, FSD, Clean Architecture, Vertical Slices



Tech Stack

Your tools **shape your architecture** — sometimes more than any pattern you choose.

Examples:

React / Vue Query — handles caching, refetching, loading states. You don't need a custom API layer or global state for server data. Your "data layer" becomes just query hooks.

Next.js / Nuxt — offer file-based routing, has opinions about pages, layouts, API routes. Usually, you work within their structure.

GraphQL + Apollo — queries live next to components, types are generated, cache is normalized. Completely different shape than REST + fetch.



Follow the tool's natural path. When a framework or library offers an approach, follow it rather than fighting against it.



Patterns

Here are the most common patterns used in Frontend Applications

UI/Components Patterns

- Container/Presentational
- Atomic Design

Feature/Structuring Patterns

- Feature-Sliced-Design
- Vertical Slices

System/Architectural Patterns

- Layered Architecture
- Clean Architecture



Patterns

Patterns can be combined with each other

- UI/component patterns can be combined with Feature and System patterns. *For example: Container/Presentation + FSD*
- Feature patterns can be combined with System patterns. *For example: Vertical slices + Layered architecture.*



Patterns

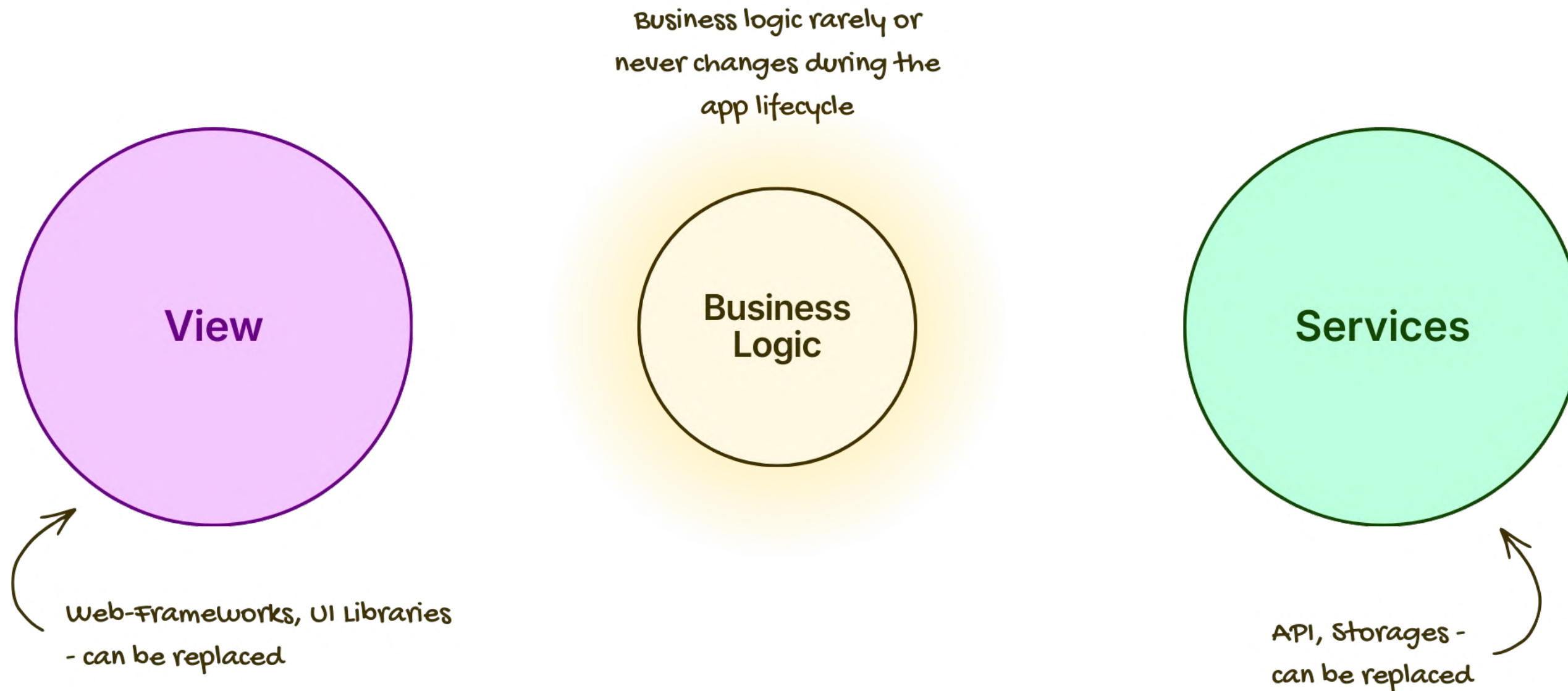
Patterns **not covered** (and why)

- **MVC, MVP patterns** — Useful to understand the evolution, but they don't help you structure a modern frontend app.
- **MVVM** — describes how frameworks like Vue work internally, not how to structure an app.
- **Hexagonal / Onion** — very similar to Clean Architecture.



What is business logic? The core rules and calculations that define how your domain works, independent of technology choices.

Business Logic





Business Logic

Business Logic:

- Calculate cart total by adding all product prices together
- User cannot checkout if cart is empty
- Premium users get 10% discount on all orders
- Maximum 5 items per product in cart

Non-business logic:

- Show error notification when checkout fails → UI/View Logic
- Make HTTP request to */api/cart* when component mounts → Infrastructure Logic
- Disable checkout button while payment is processing → UI/View Logic
- Persist cart using localStorage → Infrastructure Logic

1 Container / Presentational



Container / Presentational

Core idea: Split components by responsibility. *Presentational* components render UI. *Containers* handle logic and pass data down. *This pattern only covers component organization.*

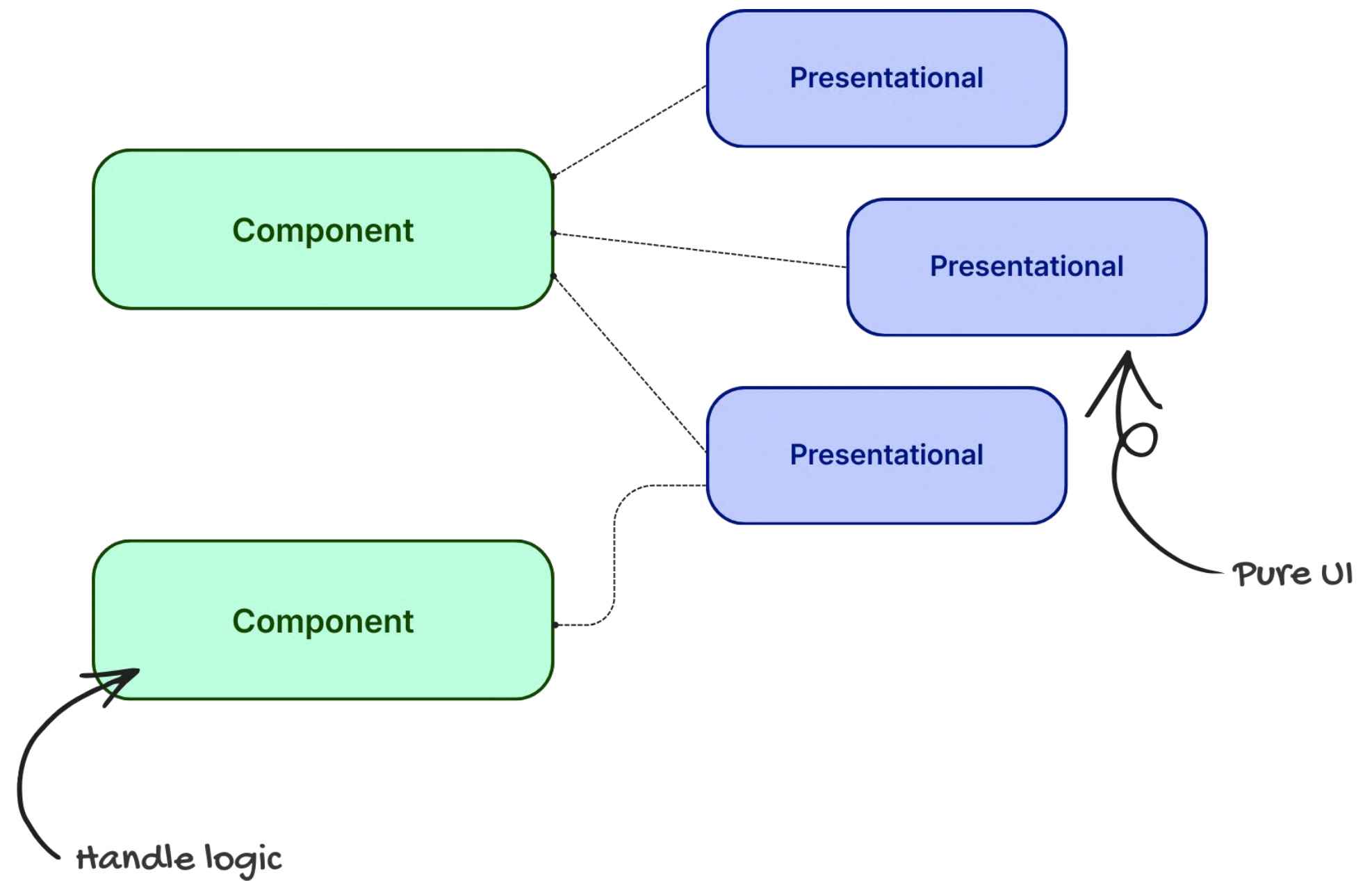
Pros:

- UI stays clean - easier to reuse and test
- Can be combined with Layered, Vertical Slices, and other patterns

Cons:

- Says nothing about where API calls, services, or business logic live.
- The "fat container" problem. All the logic piles up in containers.
- Logic can live in custom hooks now, so you don't *really* need a separate "container" component.

Container / Presentational



Container / Presentational

```
src/
├── containers/           # Logic, data fetching
│   ├── UserProfileContainer.tsx
│   ├── ProductListContainer.tsx
│   └── CartContainer.tsx
├── components/          # Pure UI, receives props
│   ├── UserProfile.tsx
│   ├── ProductCard.tsx
│   ├── CartItem.tsx
│   └── Button.tsx
├── hooks/
│   └── useUser.ts
├── api/
│   └── userApi.ts
└── shared/
    └── utils/
```

high-level
structure

```
index.ts

// containers/UserProfileContainer.tsx
export const UserProfileContainer = () => {
  const { data, isLoading } = useUser();

  if (isLoading) return <Spinner />;
  return <UserProfile user={data} />;
};

// components/UserProfile.tsx
export const UserProfile = ({ user }: Props) => (
  <div>
    <h1>{user.name}</h1>
    <p>{user.email}</p>
  </div>
);
```

Container has all logic

Container renders components

UserProfile is a pure UI component



Container/Presentational is still useful, but more as a *tactic* than a global architecture rule. You might apply it to specific parts of the app rather than using it as your *main architecture*.

Use it for

- Complex or growing screens with a lot of logic
- Heavy reuse of the same UI with different data sources
- Easier unit testing — keeping components as pure UI makes them simpler to test

Don't use it for

- Small components or simple pages — keep everything in one component or use hooks
- Blindly applying it to the whole app

2 Layered Architecture



Layered Architecture

Core idea: Organize code into horizontal layers. Dependencies flow down, never up. *Lower layers don't know about higher layers.*

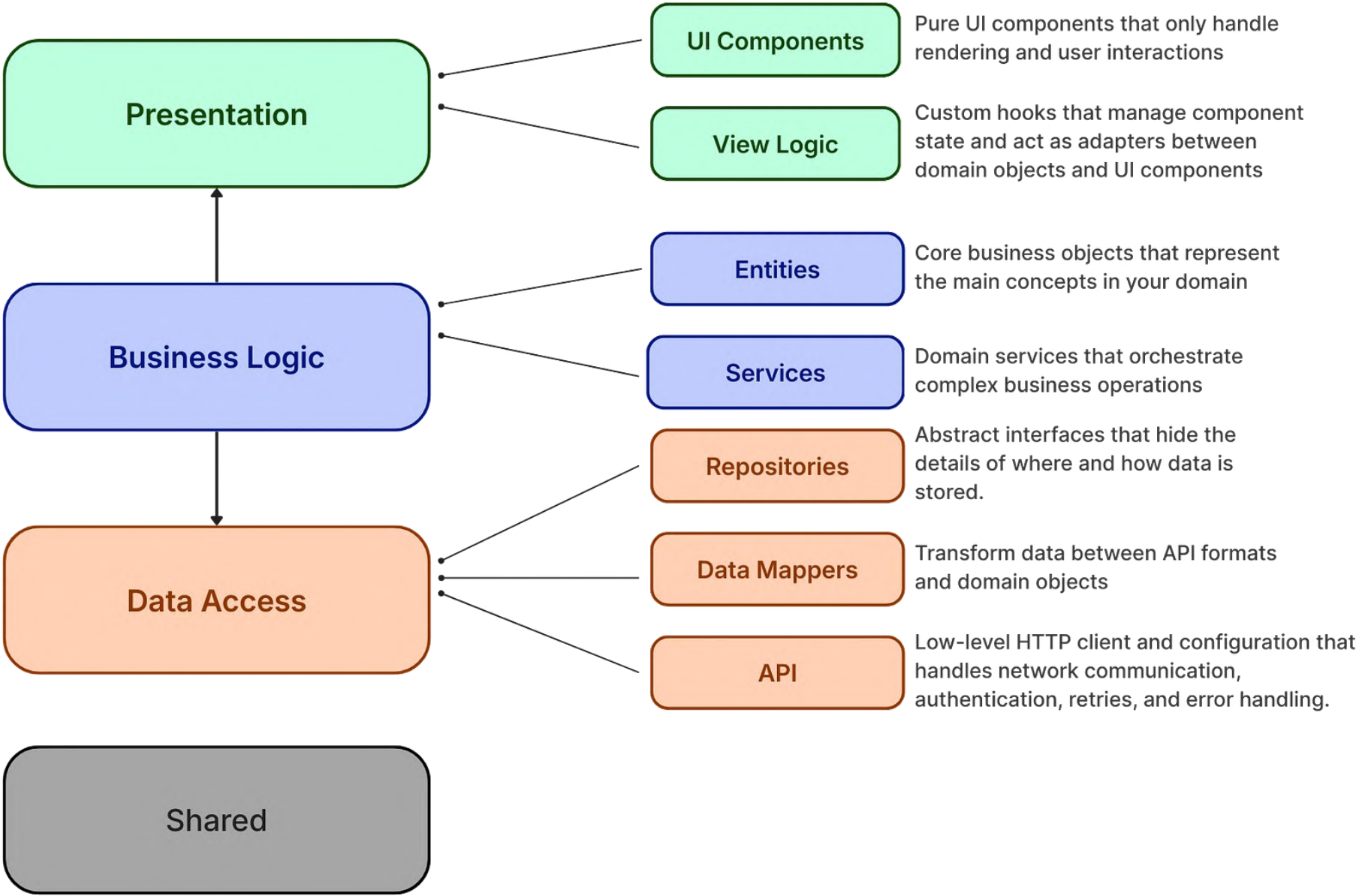
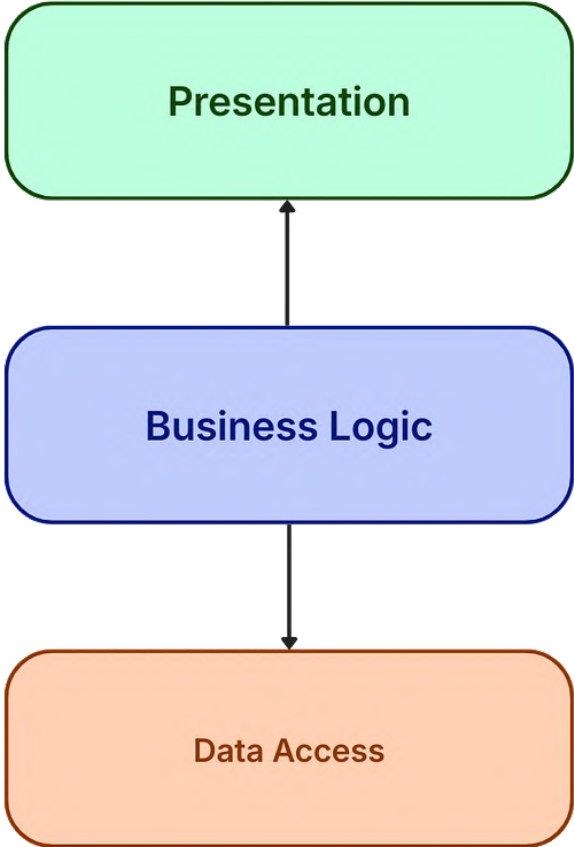
Pros:

- Simple mental model - easy to understand and explain to the team
- Minimal upfront decisions - start building, figure out structure later
- Flexible - add or remove layers as the app grows
- Keeps business rules isolated from UI and API changes

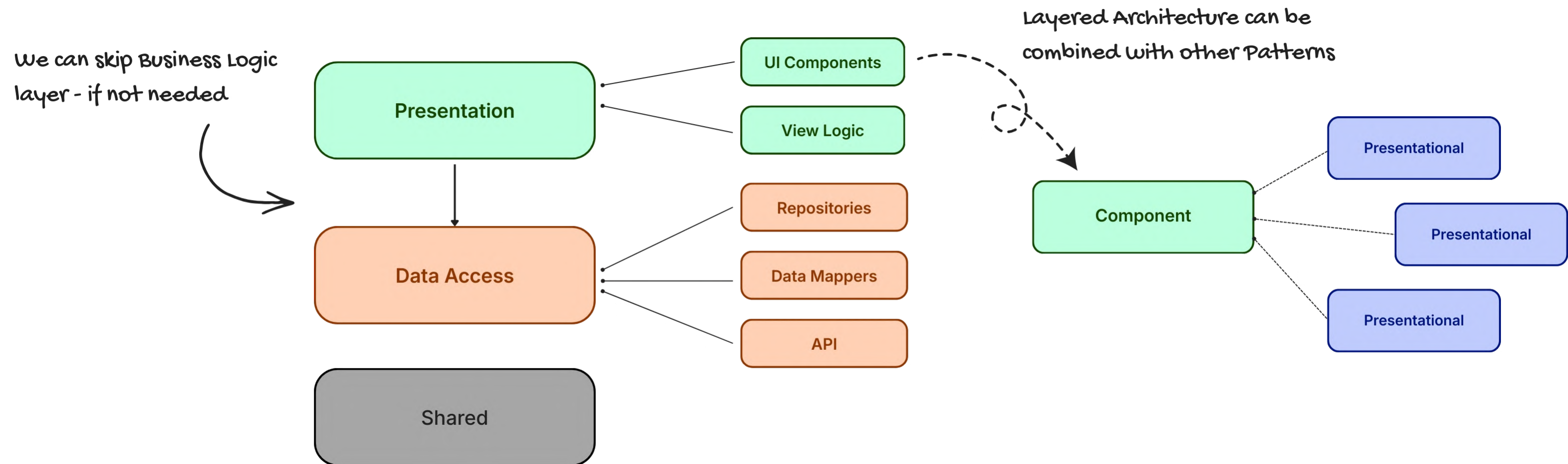
Cons:

- Learning curve.
- No feature boundaries.
- Doesn't scale for large teams.

Layered Architecture



Layered Architecture (No Business Logic) + Container / Presentational



Layered Architecture. Example

```
// services/cartService.ts (Business Logic Layer)
import { cartApi } from '@api/cartApi';

export const cartService = {
  getCartWithTotals: async () => {
    const cart = await cartApi.fetch();
    return {
      ...cart,
      subtotal: calcSubtotal(cart.items),
      discount: calcDiscount(cart.items),
      total: calcTotal(cart.items),
    };
  },

  addItem: async (productId: string, qty: number) => {
    if (qty > 10) throw new Error('Max 10 items');
    if (qty < 1) throw new Error('Min 1 item');
    return cartApi.addItem(productId, qty);
  },
};
```

Service can be
replaced with React
hook

```
// pages/CartPage.tsx (Presentation Layer)
import { cartService } from '@services/cartService';

function CartPage() {
  const [cart, setCart] = useState<Cart | null>(null);

  useEffect(() => {
    cartService.getCartWithTotals().then(setCart);
  }, []);

  const handleAdd = (productId: string) => {
    cartService.addItem(productId, 1).then(/* refetch */);
  };

  return <CartView cart={cart} onAdd={handleAdd} />;
}
```

```
// api/cartApi.ts (Data Access Layer)
export const cartApi = {
  fetch: () => api.get<Cart>('/cart'),
  addItem: (productId: string, qty: number) =>
    api.post('/cart/items', { productId, qty }),
};
```



Layered Architecture divides the app into horizontal layers. Dependencies *always* go down - higher layers import from lower layers. Add or skip layers as needed.

Use it for:

- Medium to large apps with business rules, validations, complex workflows
- The app naturally fits into layers — you don't need to force it
- You want a simple starting point that can evolve

Don't use it when:

- Small applications — overhead isn't worth it
- App has little or no business logic
- You end up with a lot of pass-through code (sinkhole / "proxy" layers that just forward calls)

3 Vertical Slices



Vertical Slices

Core idea: Organize by feature, each slice contains everything it needs (UI, logic, API, types).

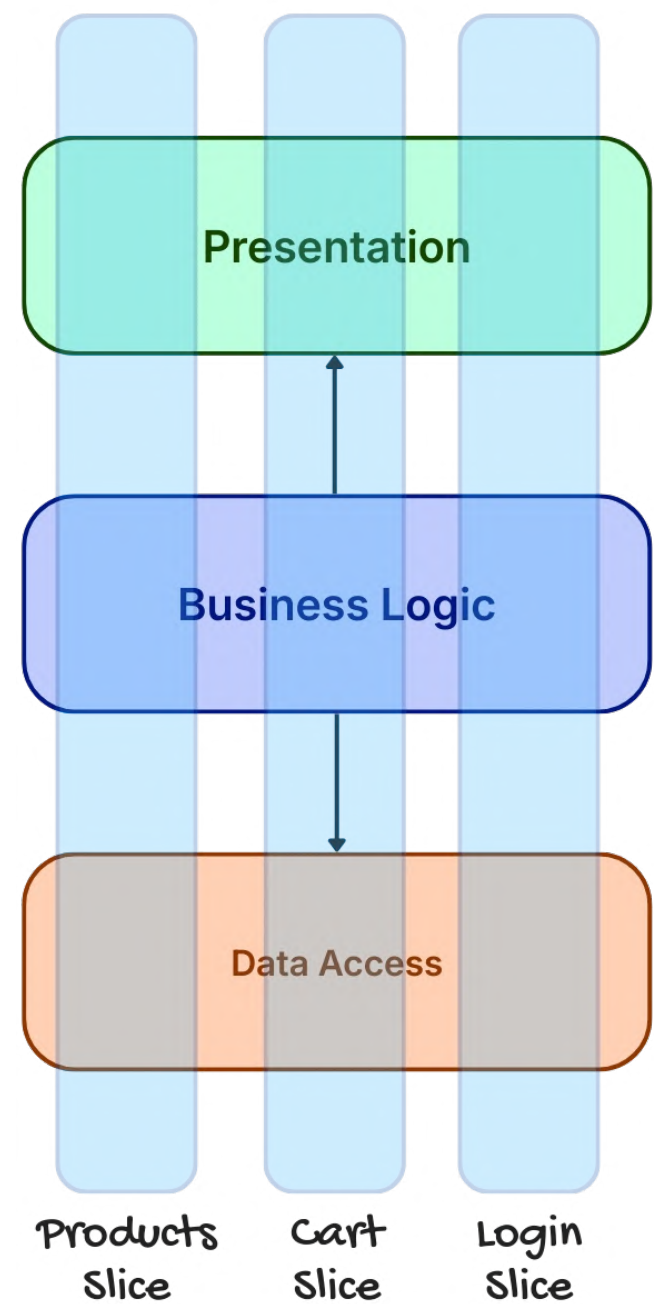
Pros:

- Scales well - large codebases stay organized
- Natural mapping - the way you talk about the product matches how code is organized

Cons:

- Learning curve.
- Shared folder might become a mess.

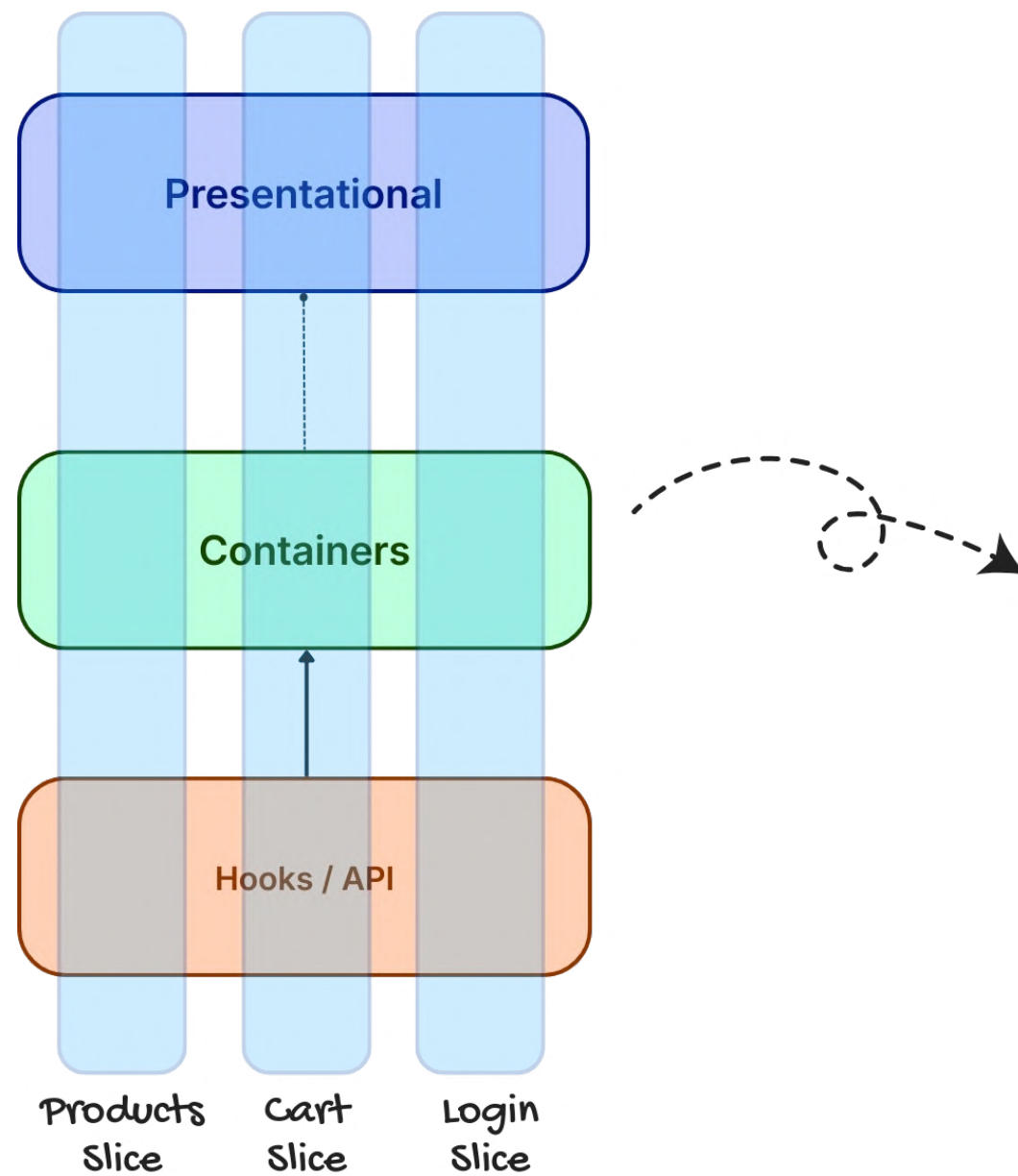
Vertical Slices + Layered Architecture



```
src/  
├── features/  
│   ├── cart/  
│   │   ├── ui/  
│   │   │   └── CartPage.tsx  
│   │   ├── services/  
│   │   │   └── cartService.ts  
│   │   ├── api/  
│   │   │   └── cartApi.ts  
│   │   └── index.ts  
│   └── products/  
│       ├── ui/  
│       │   └── ProductList.tsx  
│       ├── api/  
│       │   └── productsApi.ts  
│       └── index.ts  
├── shared/  
│   ├── ui/  
│   │   ├── Button.tsx  
│   │   └── Input.tsx  
│   ├── api/  
│   │   └── apiClient.ts  
│   └── utils/  
│       └── formatters.ts  
└── app/  
    ├── App.tsx  
    └── routes.tsx
```

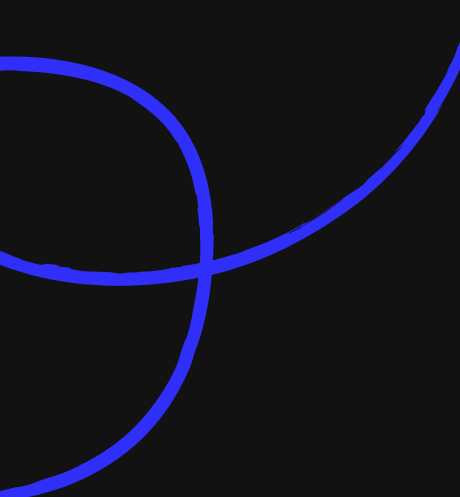
Project Structure Example

Vertical Slices + Container / Presentational



```
src/  
├── features/  
│   ├── cart/  
│   │   ├── CartPage.tsx      # Container – logic, data fetching  
│   │   ├── CartView.tsx     # Presentational – just UI  
│   │   ├── useCart.ts       # Hook – data fetching logic  
│   │   ├── cartApi.ts       # API calls  
│   │   └── index.ts  
│   └── products/  
│       ├── ProductsPage.tsx  # Container  
│       ├── ProductList.tsx   # Presentational  
│       ├── productsApi.ts  
│       └── index.ts  
├── shared/  
│   ├── ui/  
│   │   ├── Button.tsx  
│   │   ├── Input.tsx  
│   │   └── Modal.tsx  
│   └── utils/  
│       └── formatters.ts  
└── app/  
    ├── App.tsx  
    └── routes.tsx
```

Project Structure Example



Organize by feature/use-case (Auth, UserProfile, Checkout), so each slice contains everything needed for that feature: UI, logic, and data.

When to use

- Apps with many features / use-cases
- Multiple developers working on different features - need clear boundaries
- Each feature has many related files (components, hooks, API calls) -grouping them in one folder feels natural

When not to use

- Tiny apps with just a few screens - flat structure is clearer
- Features heavily share logic — you'll end up with duplication or a bloated shared folder

4 Feature-Sliced Design



Featured-Sliced Design (FSD)

Core idea: Layered Architecture + Vertical Slices + strict import rules.

- Vertical — import only from layers below
- Horizontal — files on the same layer never import each other

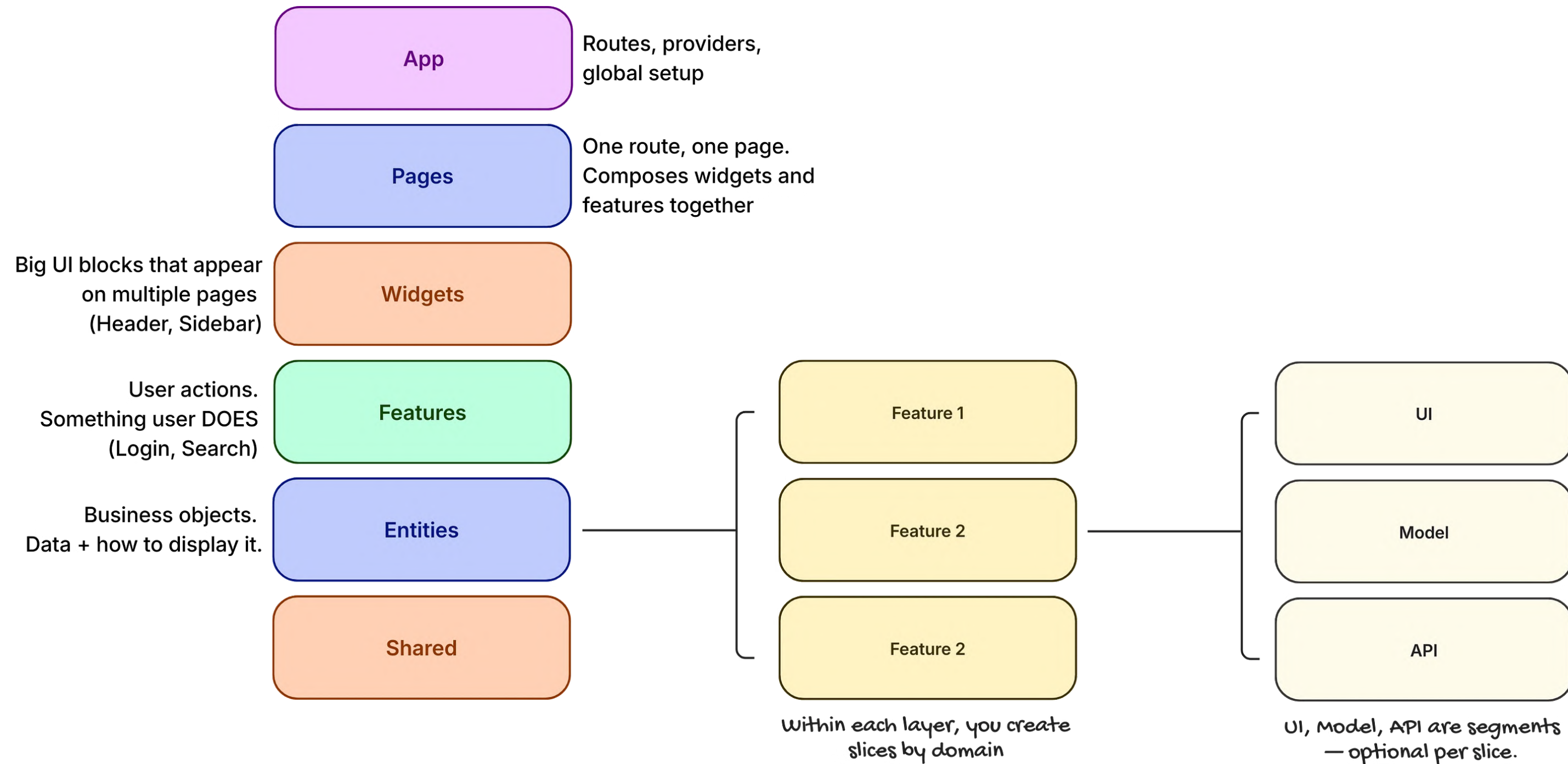
Pros:

- Strict rules
- Feature-oriented - code organization matches how product is built
- Popular in the community - many developers already know it

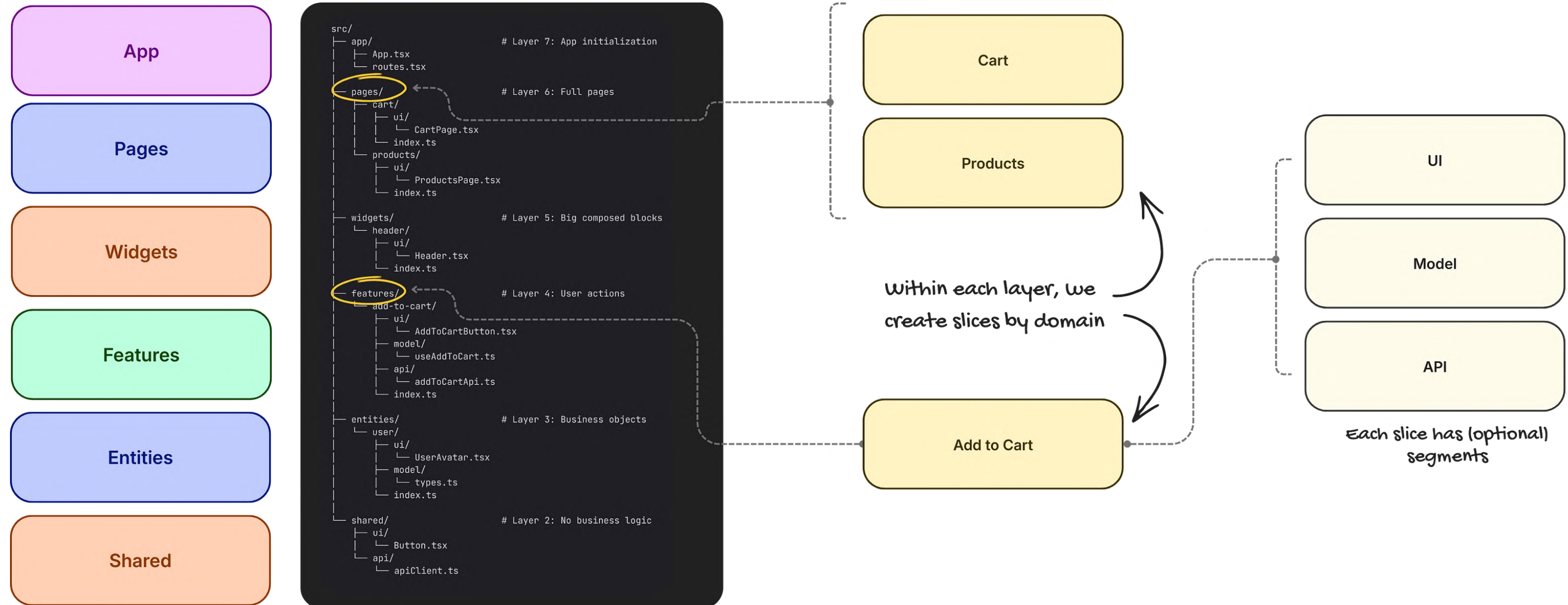
Cons:

- Learning curve.
- Low flexibility.
- Strict rules may slow down developing.
- Classifies by type, not by domain - related code lives in different places

Feature-Sliced Design



Feature-Sliced Design





FSD is a well-documented architecture pattern, very popular especially in the *React community*.

Use it if

- **Business logic is complex (finance, healthcare, e-commerce calculations)**

Don't use it if

- You're at an early stage and still figuring out what you're building
- Your project is small to medium sized
- You prefer *flexibility* over strict structure
- You don't want to spend time debating where things go (Widget vs Feature vs Entity)
- Your project is a focused, single-purpose service rather than a multi-feature app

5 Atomic Design



Atomic Design

Core idea: UI components in a hierarchy: Atoms → Molecules → Organisms → Templates → Pages. Each level builds on the previous one.

Pros:

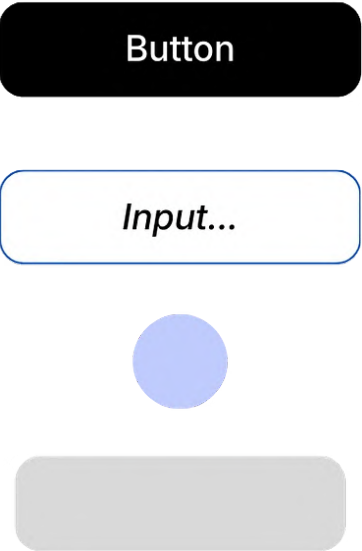
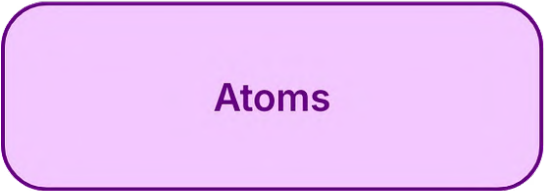
- Strict hierarchy
- Shared vocabulary with design team (if they also use Atomic Design)

Cons:

- Blurry boundaries - "Is this a molecule or organism?" is subjective and wastes time.
- Only covers UI - says nothing about business logic, API, or state.
- Premature splitting - you create atoms/molecules you don't actually need.
- Scatters related code - components that work together end up in different folders.

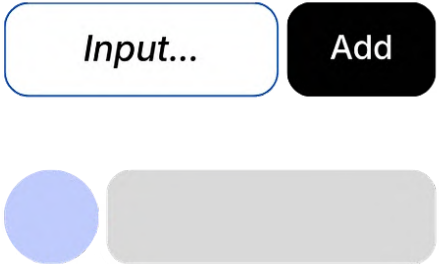
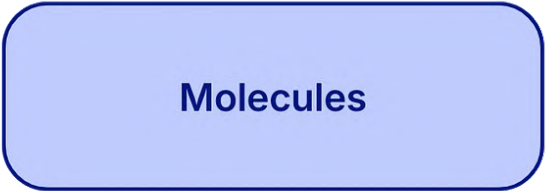
Atomic Design

Smallest, no dependencies
(Button, Input)



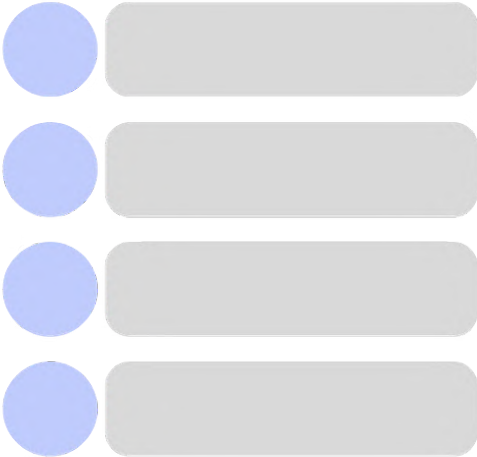
No imports allowed

Atoms combined
(SearchBar)



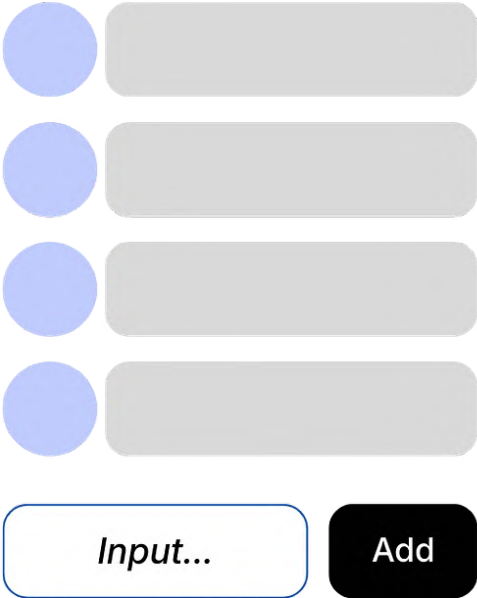
Import atoms

Molecules combined
(Header, List)



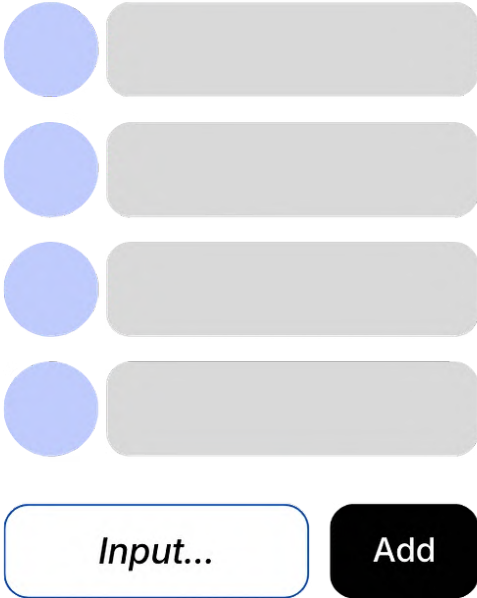
Import atoms and
molecules

UI without real data
(Form)



Import atoms,
molecules and
organisms

Template + real content
(FormPage)



Can import
anything

Atomic Design

Atoms

Molecules

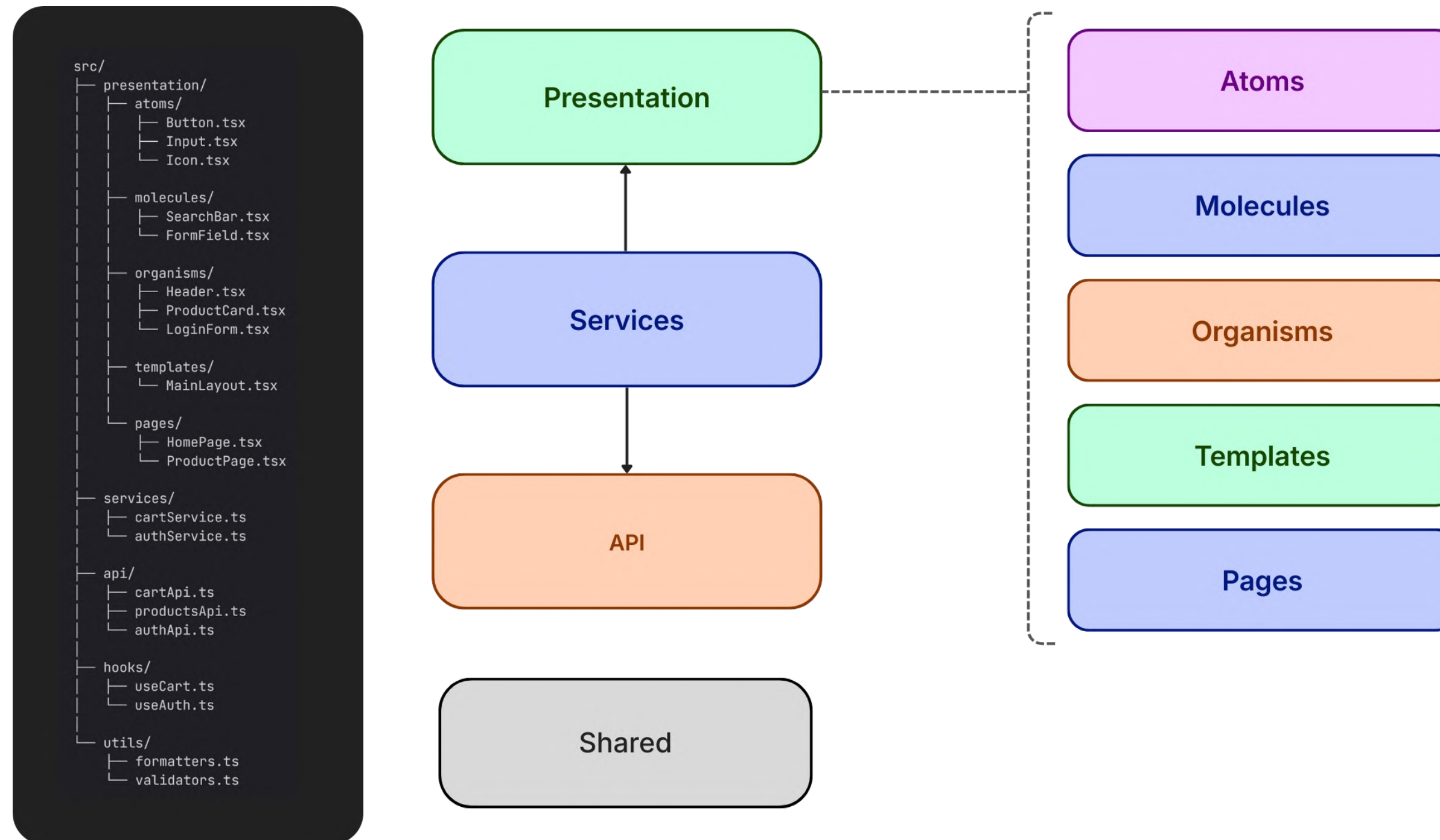
Organisms

Templates

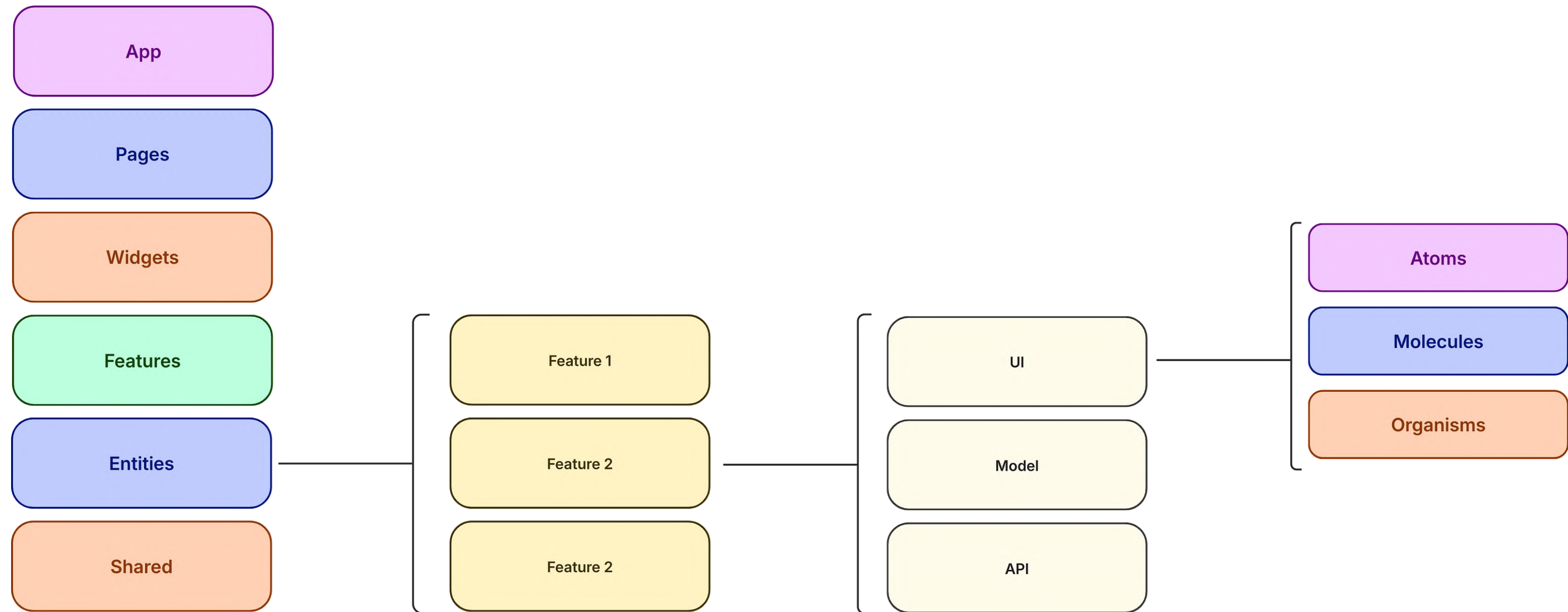
Pages

```
src/
├── components/
│   ├── atoms/
│   │   ├── Button.tsx
│   │   ├── Input.tsx
│   │   ├── Label.tsx
│   │   ├── Icon.tsx
│   │   └── Spinner.tsx
│   ├── molecules/
│   │   ├── SearchBar.tsx      # Input + Button
│   │   ├── FormField.tsx     # Label + Input + Error
│   │   ├── NavLink.tsx       # Icon + Text
│   │   └── ProductPrice.tsx   # Label + Price + Currency
│   ├── organisms/
│   │   ├── Header.tsx        # Logo + NavLinks + SearchBar
│   │   ├── ProductCard.tsx   # Image + Title + Price + Button
│   │   ├── LoginForm.tsx     # FormFields + Button
│   │   └── CartSummary.tsx    # ProductList + Total + Button
│   ├── templates/
│   │   ├── MainLayout.tsx     # Header + Content + Footer slots
│   │   ├── AuthLayout.tsx     # Centered card layout
│   │   └── DashboardLayout.tsx # Sidebar + Main content
│   └── pages/
│       ├── HomePage.tsx      # MainLayout + specific content
│       ├── ProductPage.tsx
│       ├── CartPage.tsx
│       └── LoginPage.tsx
├── services/                  # ← Atomic Design doesn't cover this
├── api/                       # ← or this
├── hooks/                     # ← or this
└── utils/
```

Atomic Design + Layered Architecture



Atomic Design + FSD





Atomic is a design pattern, not an application architecture. It only covers UI components.

Use it for

- Building a design system or component library
- Design team already uses Atomic Design in Figma
- Documenting components in Storybook

Don't use it for

- As your main application architecture
- Components aren't reused much - most are feature-specific, not shared

6 Clean Architecture



Clean Architecture

Core idea: Separate what your app DOES (business rules) from HOW it does it (React, APIs, database). Business rules don't depend on anything external.

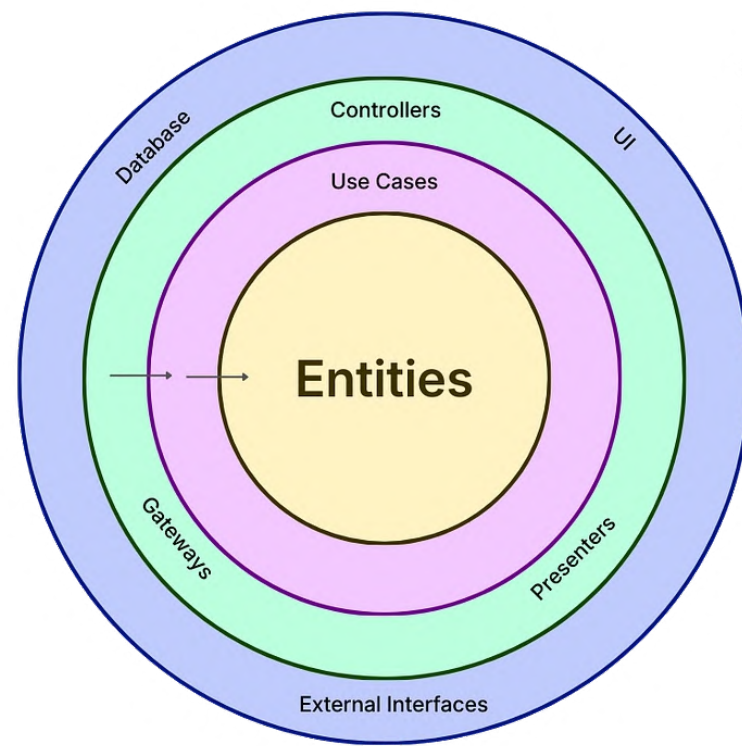
Pros:

- Strict rules
- Keeps business rules isolated from UI and API changes
- Makes testing business logic easier

Cons:

- Adds extra code that might slow development
- Takes time to learn the layers

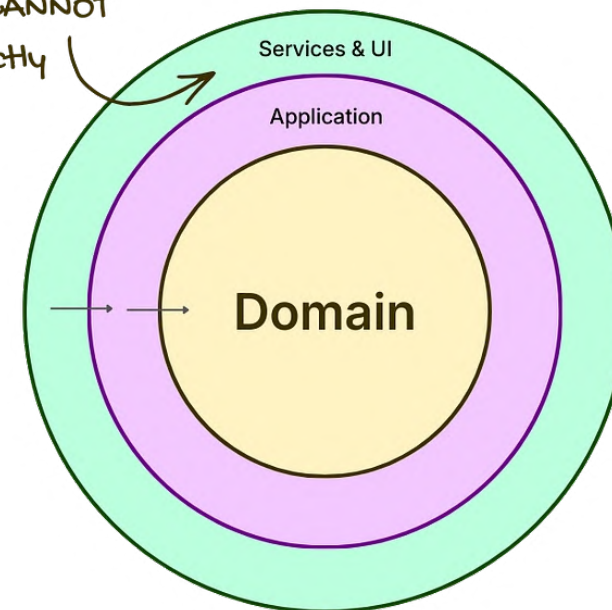
Original Clean Architecture



- **Entities** - Core business objects and rules
- **Use Cases** - *Application-specific* workflows that orchestrate entities
- **Interface Adapters** - Data format converters between your business logic and external systems
- **Frameworks & Drivers** - Third-party libraries and tools

Simplified *Frontend* Version

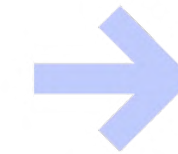
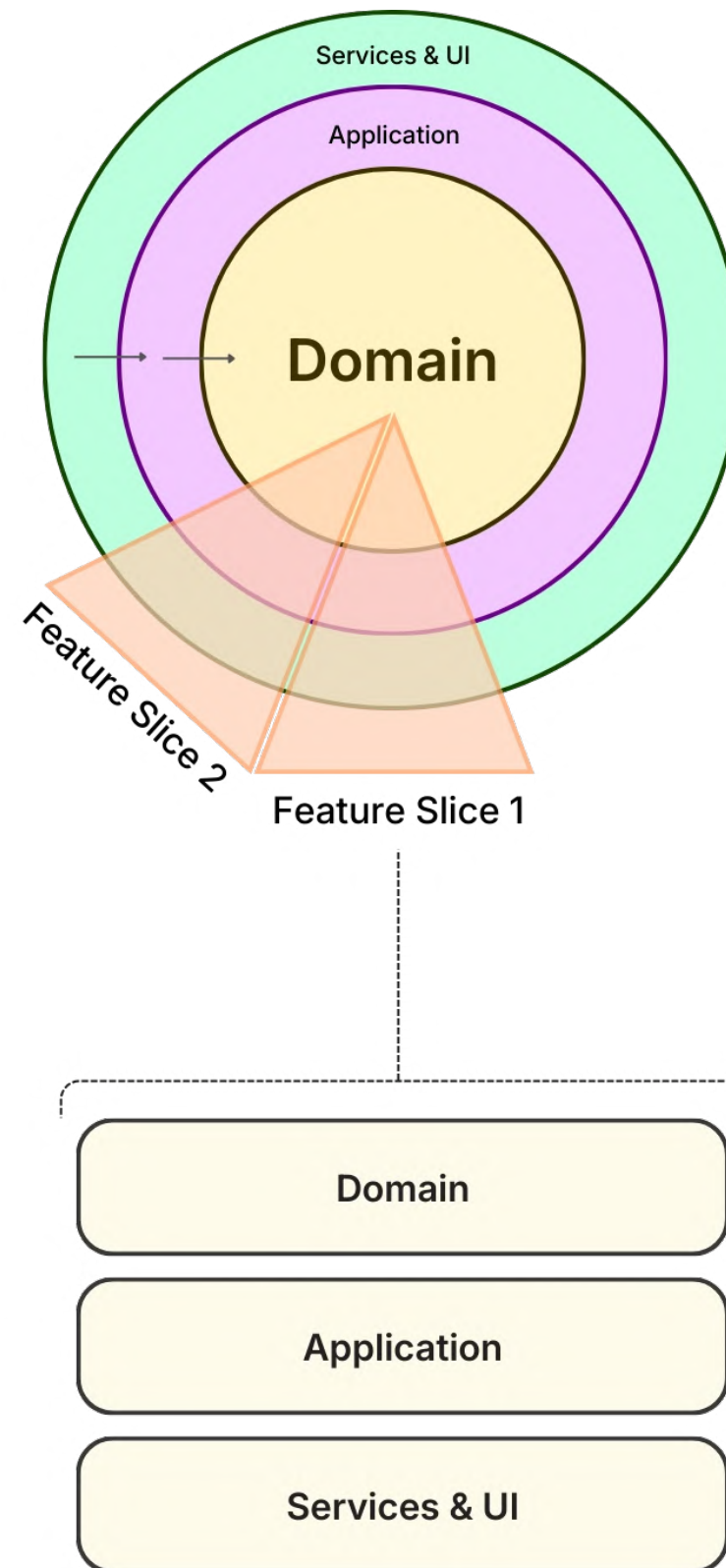
Services and UI CANNOT use Domain directly



- **Domain** - Core business objects and rules
- **Application** - Application's use cases that coordinate domain logic
- **Adapters** - Concrete service implementations (API, UI)

```
src/
├── domain/
│   ├── user.ts           # User entity types and transformations
│   ├── product.ts        # Product entity and price calculations
│   ├── order.ts          # Order entity and creation logic
│   └── cart.ts           # Cart entity and item management
├── application/
│   ├── addToCart.ts      # Use case for adding items
│   ├── authenticate.ts   # Authentication use case
│   ├── orderProducts.ts  # Checkout use case
│   └── ports.ts          # Interfaces for external services
├── services/             # Adapters layer
│   ├── authAdapter.ts
│   ├── notificationAdapter.ts
│   ├── paymentAdapter.ts
│   ├── storageAdapter.ts
│   ├── api.ts
│   └── store.tsx         # React Context for state
├── lib/                  # Shared utilities
└── ui/                   # React components (driving adapters)
```

Clean Architecture + Vertical Slices

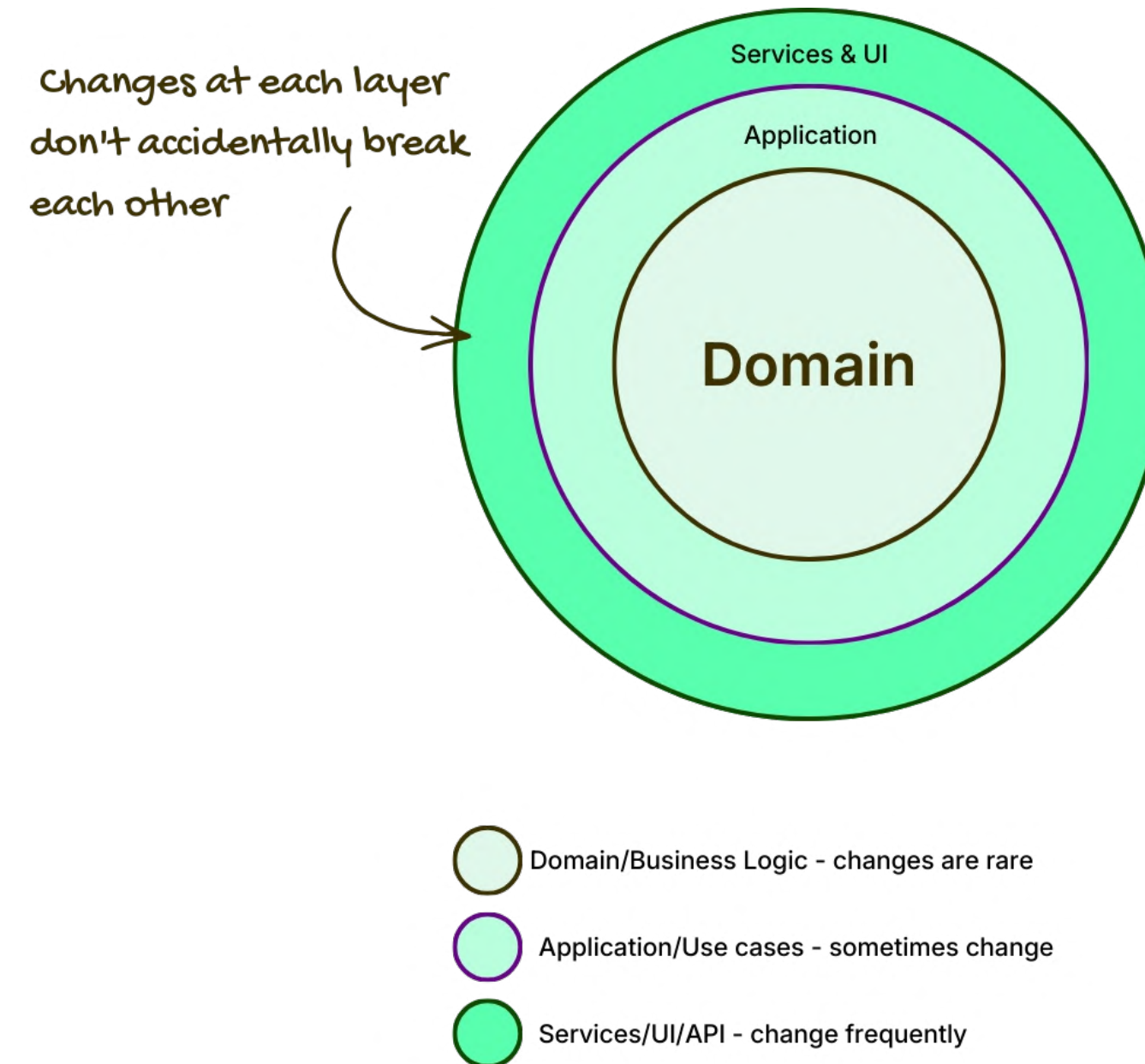


```
src/  
├── features/  
│   ├── products/  
│   │   ├── domain/  
│   │   │   ├── Product.ts  
│   │   │   └── Cart.ts  
│   │   ├── application/  
│   │   │   ├── useCases/  
│   │   │   │   └── addToCart.ts  
│   │   │   └── ports.ts  
│   │   ├── adapters/  
│   │   │   ├── ui/  
│   │   │   │   ├── ProductCard.tsx  
│   │   │   │   └── useProducts.ts  
│   │   │   └── storage/  
│   │   │       └── cartStorage.ts  
│   │   └── index.ts  
│   ├── checkout/  
│   │   ├── domain/  
│   │   ├── application/  
│   │   └── adapters/  
├── shared/  
│   ├── ui/  
│   └── utils/  
└── app/  
    └── App.tsx
```

Clean Architecture

Benefits of Isolation

- **UI changes independence** - Redesign UI without touching business logic
- **API independence** - Migrate REST → GraphQL without touching business rules
- **Easy testing** - Test calculations without heavy mocking
- **Multi-platform reuse** - Same logic on web/mobile/desktop





Clean Architecture isolates business logic at the center with zero external dependencies. Dependencies flow *inward* only.

Use it when

- Business logic is complex (finance, e-commerce calculations, healthcare)
- Multiple UIs with shared logic (React web + Vue admin + React Native mobile)
- UI and/or use-cases change frequently, logic is basically the same

Don't use it for

- Simple apps, MVPs
- Frontend is just a thin UI layer (no business logic)

Learn the
decision-making
framework



Advanced Frontend
Course

